



FIT3159 Computer Architecture

Semester 1 2026

Laboratory #5

Topic: Assembly Code Programming Interrupt Service Routines and Context Switching (3 hr)

Marks: Each lab is worth 5% of the final mark.

Interrupt Service Routines and Context Switching

Purpose of this Laboratory Exercise

The purpose of this lab is for you to gain more experience with:

- Understand ISRs
- MIPS assembly opcodes
- MIPS assembly registers
- MIPS assembler directives
- Using SPIM

This lab has a preparation component which should be attempted at home, before you start the lab. The lab tasks cannot be completed in time without the preparation work. Please budget sufficient time for this activity, and note that this assessment is a substitute for assignments in other units.

References:

1. James Larus, Appendix A: Assemblers, Linkers, and the SPIM Simulator (on Moodle);
2. FIT3159 Lecture slides.

Task 1 (0% Reading Only)

Interrupt Service Routines (ISR) are one of the most widely used constructs in operating systems and other low level utility code. ISRs are employed to service interrupts in device drivers, but also used to handle a great many other functions where an interrupt is employed to switch the thread of execution in the processor.

As an interrupt handler by definition involves breaking the existing thread of execution, to allow the ISR code to be executed, an essential part of running an ISR is saving the state of the CPU before the ISR changes any registers, and upon completion of the ISR code, restoring the state of the CPU so the previous executing code can be resumed immediately.

How is an ISR constructed? Let us first consider what things an ISR must do when the CPU receives an interrupt:

1. Disable interrupts so the ISR can execute without being pre-empted;
2. Save all of the CPU registers which might be changed;
3. Execute whatever work the ISR is intended to perform;
4. Restore all of the registers which have been saved;

5.Enable interrupts

6.Transfer control of the CPU back to the code executing before the ISR was called.

Most of these steps are usually coded in assembler, even if the body of the ISR is coded in C language. Disabling and enabling interrupts is usually performed using a pair of CPU specific instructions.

The manner in which CPU registers are saved, and which registers are saved, is also specific to the type of CPU. MIPS processors use a unique method in which some registers are saved directly to memory locations, and the remainder are saved to a stack.

Stacks are the most commonly used location for saving program context information (i.e. CPU state) when an ISR is being executed. Many operating systems will maintain a dedicated stack which is used by the operating system kernel for handling interrupts, while a separate stack is used for function calls. Simpler embedded systems will usually employ a single stack for everything.

The purpose of this laboratory exercise is to produce a short piece of code in assembly language which emulates the behaviour of an ISR. You will then insert this code into the middle of an existing piece of assembly code to test whether your code is properly saving and restoring the state of the CPU. The existing piece of assembly code you have produced in Lab 4 should be used for testing.

Task 2 Emulating an ISR (Preparation + 60% Laboratory)

Attempt all Task 2 at home before you enter the Lab. There may not be sufficient time in the Lab to complete Task 2 without preparation.

The starting point in emulating an ISR is that you need a "runtime environment" for the ISR to save CPU state information. Using your existing code, add in the following:

- 1.A reserved 512 word area of memory for use as an interrupt stack - MIPS processors usually run with two separate stacks, named `stack` and `kstack`, and SPIM uses only a `stack`, but since we are doing an emulation we will reserve memory to pretend we are using a `kstack`.

Having set up the pretend stack as `kstack:`, we can now start on our program proper.

In the `.text` segment, set up an area for your code, before the exit section of the existing code, with the label `pseudoisr:`. When you are testing your code, you will need to jump to `pseudoisr`, execute the code in `pseudoisr`, and then let `pseudoisr` return to the location after the jump instruction.

What will `pseudoisr:` actually look like?

`pseudoisr:`

- 1.Pretend to disable interrupts using a comment;
- 2.Save all of the CPU registers which might be changed to `kstack`;
- 3.Execute whatever work the ISR is intended to perform;
- 4.Restore all of the registers which have been saved from `kstack`;
- 5.pretend to enable interrupts using a comment;
- 6.Jump back to location following the ISR.

The task for this laboratory is to write the body of `pseudoisr:` and

then test it. Perform the following steps.

- 1.Add the pretend interrupt disabling and enabling instructions at the start and end of `pseudoisr` as comments;
- 2.Using `$t7` as a pretend `kstack` pointer, "push" the contents of registers `$t0` - `$t5` to `kstack`, this will require you to store the contents of each of these registers to `kstack`, and then increment the pretend `kstack` pointer;
- 3.Set registers `$t0` - `$t6` to distinct integer values (e.g., `$t0 = 5`, `$t1 = 10`, ..., `$t6 = 35`).
- 4.Using the registers `$t0` - `$t6`, compute the average of the values in `$t0` - `$t6` using a simple loop or series of additions and a division. Store the result in register `$s1`.
- 5.Again, using `$t7` as a pretend `kstack` pointer, "pop" the saved register values off `kstack`, to

restore their contents, do not forget that the pretend `kstack` pointer must be decremented with each "pop";

6. Add the jump instruction to `returnfrompseudoisr` at the end of `pseudoisr` to return to the location you called from.

To test the operation of `pseudoisr`, we require a jump instruction to `pseudoisr`, followed by a label to return to. These two lines are then inserted into any code, to test the operation of `pseudoisr`:

```
j pseudoisr
returnfrompseudoisr:
```

The first stage of the testing process is simple, as we simply need to single-step through the code to verify that `pseudoisr` is working as intended, and that the register values are appearing on `kstack`, and then reappearing in the registers as we "pop" them off `kstack`.

The second stage of the testing process is that we change the location of jump to `pseudoisr` to see if the code is still behaving. The code which `pseudoisr` has been inserted into should behave as before the addition of `pseudoisr`. Try three different locations in your code.

If your original code used `$t7` for another purpose, you can make use of any unused register as a pretend `kstack` pointer.

Note that we are not using the SPIM stack as the purpose of this exercise is to perform the setup and operation of a generic ISR and generic stack, with no convenient shortcuts.

Assessment criteria:

- Full marks if all code works and student can show understanding of what the code does;
- Partial marks if code works with bugs;
- Zero marks if code does not work.

Task 3 Estimating ISR Execution Time (Preparation + 40% Laboratory)

Assembly code is at this time mostly employed in situations, where speed is very important, and very short execution time for the code is critical. This is because the code can be optimised or tuned to execute in as short a time as possible, by careful choices in instructions, and careful design of the algorithm to be used. Instances of code that are commonly written in assembly language are algorithms which are executed at very high frequencies (number of times per second), in operating systems, embedded code, and code for multimedia libraries or applications. In operating systems, code used in the kernel for context switching and in device drivers, often includes assembly code routines. ISRs, which are usually incorporated in a device driver, are very often written in assembly code.

How can we estimate the time it takes for a fragment of assembly code to execute?

As you have learned across a number of FIT3159 lectures, the execution of an instruction takes place over several machine clock cycles. How many clock cycles depends both on the architecture of the processor core, and how the core has been designed internally. In general, the less work the instruction has to perform on its operands, the fewer clock cycles are required. If more work is performed, more clock cycles are required.

Consider a RISC processor that runs at a clock frequency of 1 GigaHertz. Each clock cycle has a duration of 1 nanosecond:

$$f_{\text{core}} = 1 \times 10^9 \text{ [Hz];}$$

$$t_{\text{cycle}} = 1 / f_{\text{core}};$$

$$t_{\text{cycle}} = 1 / 1 \times 10^9 = 1 \times 10^{-9} \text{ [sec]} = 1 \text{ [ns];}$$

A short instruction such as a copy from register to register might require 1 clock cycle, and will take 1 nanosecond to execute. A complex instruction such as an integer multiply between two registers might take 10 clock cycles, or 10 nanoseconds to execute. We assume the instruction is in cache, and all

instructions are prefetched, so we do not have to count the opcode fetch cycle in the execution time.

An example for a RISC architecture like the MIPS might be thus:

Instruction Type	Clock Cycles per Instruction	Notes
Load	5	
Store	4	
Register Operation	2	e.g. move operand between registers
Simple Integer Arithmetic Operation	4	e.g. add, sub, shift operands
Complex Integer Arithmetic Operation	32 - 64	e.g. multiply, divide
Branch	3	
Jump	3	

Table 1. Example Clock Cycle Durations for Instructions

If the core is running at a slower clock frequency, such as 500 MHz, each clock cycle will take twice as long, so the instruction execution times have to be multiplied by two. If the core is running at a higher clock frequency, such as 3 GHz, each clock cycle will take one third of the time to execute, so the instruction execution times have to be divided by three.

The method for estimating how long it takes for a fragment of code to execute has been used by programmers in industry for decades. It is very simple - tabulate all of the instructions in the code, include in the table a column for the number of clock cycles each instruction takes to execute, and then add up the total number of clock cycles required to execute the code fragment. Then multiply the total number of clock cycles by the duration of one clock cycle.

For Task 3, you need to produce a table for your `pseudoisr` code fragment, and estimate the duration of its execution, for the following parameters, assuming cycle values in Table 1:

	CPU Core Clock Frequency	Example CPU Chip / SOC	Release Year
Case A	400 MHz	ARM Cortex-A5	2010
Case B	1.2 GHz	Qualcomm Snapdragon 410	2014
Case C	2.84 GHz	Qualcomm Snapdragon 888	2021
Case D	3.78 GHz (performance core)	Apple M2 Pro	2023
Case E	5.3 GHz (turbo boost)	Intel Core i9-13900K	2022

Create a document that includes an Excel sheet demonstrating your calculations and provide an explanation of the results you have obtained.

Assessment criteria:

- Full marks if calculation is correct and student can show understanding of how the result was produced;
- Partial marks if calculation has some errors or student cannot show understanding of how the result was produced;
- Zero marks if calculation errors are an order of magnitude different from the correct answer, or no work submitted;

Submit your files to Moodle before you leave the lab.